



GPU and TPU

Carlos Cotrini



Section 1

From games to matrices

1

AlexNet, 2012

The result that started all of this was trained on two graphics cards bought for playing games.

AlexNet, 2012

The result that started all of this was trained on two graphics cards bought for playing games.

GeForce GTX 580
3 GB

GeForce GTX 580
3 GB

AlexNet, 2012

The result that started all of this was trained on two graphics cards bought for playing games.

GeForce GTX 580
3 GB

GeForce GTX 580
3 GB

Six days. About 60 million parameters, split across the two cards because one card's memory could not hold it.

AlexNet, 2012

The result that started all of this was trained on two graphics cards bought for playing games.

GeForce GTX 580
3 GB

GeForce GTX 580
3 GB

Six days. About 60 million parameters, split across the two cards because one card's memory could not hold it.

Nothing about them was designed for this

AlexNet, 2012

The result that started all of this was trained on two graphics cards bought for playing games.

GeForce GTX 580
3 GB

GeForce GTX 580
3 GB

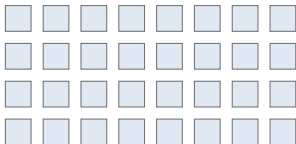
Six days. About 60 million parameters, split across the two cards because one card's memory could not hold it.

Nothing about them was designed for this

Krizhevsky, Sutskever and Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, [NeurIPS 2012](#). No arXiv version exists.

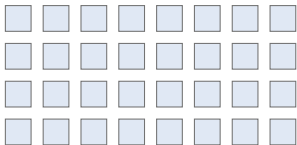
A screen and a matrix want the same thing

Shading a screen



A screen and a matrix want the same thing

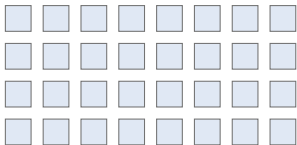
Shading a screen



Every pixel gets the
same short recipe

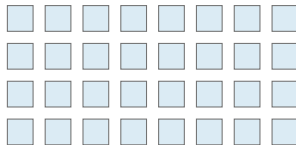
A screen and a matrix want the same thing

Shading a screen



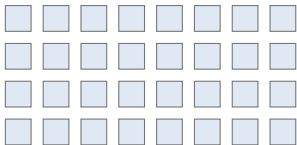
Every pixel gets the
same short recipe

Multiplying a matrix



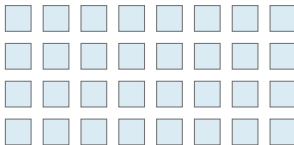
A screen and a matrix want the same thing

Shading a screen



Every pixel gets the
same short recipe

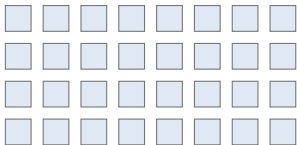
Multiplying a matrix



Every output cell gets the
same short recipe

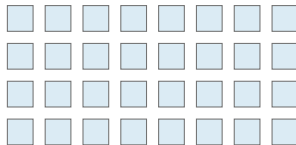
A screen and a matrix want the same thing

Shading a screen



Every pixel gets the
same short recipe

Multiplying a matrix



Every output cell gets the
same short recipe

- Thousands of identical, independent little jobs. That is the shape a graphics card was already built for



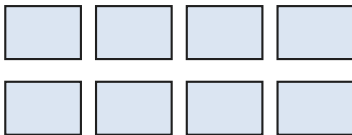
Section 2

Inside the box

2

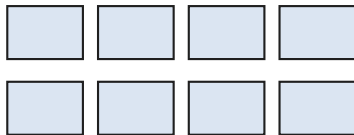
Tens of cores, or seventeen thousand

The Grace CPU



Tens of cores, or seventeen thousand

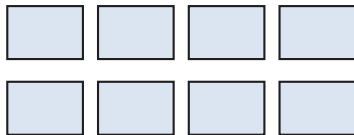
The Grace CPU



72 cores, each one clever

Tens of cores, or seventeen thousand

The Grace CPU



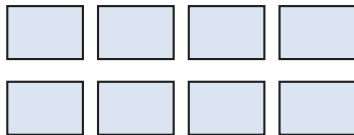
72 cores, each one clever

The Hopper GPU



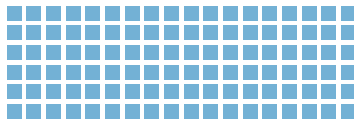
Tens of cores, or seventeen thousand

The Grace CPU



72 cores, each one clever

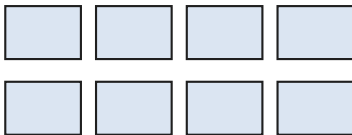
The Hopper GPU



16,896 cores, each one simple

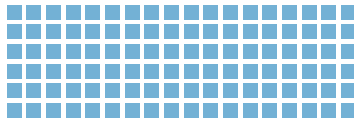
Tens of cores, or seventeen thousand

The Grace CPU



72 cores, each one clever

The Hopper GPU



16,896 cores, each one simple

235 times as many

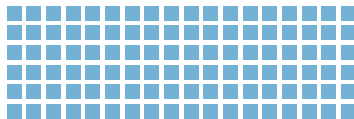
Tens of cores, or seventeen thousand

The Grace CPU



72 cores, each one clever

The Hopper GPU



16,896 cores, each one simple

235 times as many

- The CPU is built to do one hard thing quickly. The GPU is built to do one easy thing many thousands of times at once

They all have to do the same thing

One instruction, issued to a whole group



They all have to do the same thing

One instruction, issued to a whole group



Give them a branch, and the ones on the wrong side wait

They all have to do the same thing

One instruction, issued to a whole group



Give them a branch, and the ones on the wrong side wait



They all have to do the same thing

One instruction, issued to a whole group



Give them a branch, and the ones on the wrong side wait



Half the machine idle, on work that a CPU would not blink at

They all have to do the same thing

One instruction, issued to a whole group



Give them a branch, and the ones on the wrong side wait



Half the machine idle, on work that a CPU would not blink at

- So a GPU is superb at matrices and poor at decisions, which is why the CPU on the module reads the files and runs the Python

The tensor cores do the matrix work

16,896 FP32 cores
the general purpose ones

The tensor cores do the matrix work

16,896 FP32 cores
the general purpose ones

528 tensor cores
they do nothing but
small matrix products

The tensor cores do the matrix work

16,896 FP32 cores
the general purpose ones

528 tensor cores
they do nothing but
small matrix products

The 990 TFLOP/s you divided by at 08:00 is the tensor cores alone

The tensor cores do the matrix work

16,896 FP32 cores
the general purpose ones

528 tensor cores
they do nothing but
small matrix products

The 990 TFLOP/s you divided by at 08:00 is the tensor cores alone

Each tensor core takes two small matrices and a running total, and returns the product added to the total, in one instruction.

The tensor cores do the matrix work

16,896 FP32 cores
the general purpose ones

528 tensor cores
they do nothing but
small matrix products

The 990 TFLOP/s you divided by at 08:00 is the tensor cores alone

Each tensor core takes two small matrices and a running total, and returns the product added to the total, in one instruction.

- A multiply and an add, which is the multiply accumulate from 08:00, done on whole tiles at a time

Where 990 TFLOP/s comes from

528
tensor cores

Where 990 TFLOP/s comes from

$$\underbrace{528}_{\text{tensor cores}} \times \underbrace{1024}_{\text{FLOP per cycle}}$$

Where 990 TFLOP/s comes from

$$\underbrace{528}_{\text{tensor cores}} \times \underbrace{1024}_{\text{FLOP per cycle}} \times \underbrace{1.83 \text{ GHz}}_{\text{clock}}$$

Where 990 TFLOP/s comes from

$$\underbrace{528}_{\text{tensor cores}} \times \underbrace{1024}_{\text{FLOP per cycle}} \times \underbrace{1.83 \text{ GHz}}_{\text{clock}} = 989 \text{ TFLOP/s}$$

Where 990 TFLOP/s comes from

$$\underbrace{528}_{\text{tensor cores}} \times \underbrace{1024}_{\text{FLOP per cycle}} \times \underbrace{1.83 \text{ GHz}}_{\text{clock}} = 989 \text{ TFLOP/s}$$

- Peak is counted, never measured. This is the counting

Where 990 TFLOP/s comes from

$$\underbrace{528}_{\text{tensor cores}} \times \underbrace{1024}_{\text{FLOP per cycle}} \times \underbrace{1.83 \text{ GHz}}_{\text{clock}} = 989 \text{ TFLOP/s}$$

- Peak is counted, never measured. This is the counting
- The 1024 is 512 multiply accumulates, each worth two operations

Where 990 TFLOP/s comes from

$$\underbrace{528}_{\text{tensor cores}} \times \underbrace{1024}_{\text{FLOP per cycle}} \times \underbrace{1.83 \text{ GHz}}_{\text{clock}} = 989 \text{ TFLOP/s}$$

- Peak is counted, never measured. This is the counting
- The 1024 is 512 multiply accumulates, each worth two operations

The clock here is reverse engineered from the published rate, so treat the shape as the lesson: units times clock times two.



Section 3

The other answer

3

A TPU is built for one job

Google built its own chip because it could see how much of its datacentre was about to be doing one thing.

A TPU is built for one job

Google built its own chip because it could see how much of its datacentre was about to be doing one thing

A GPU

a general machine that
happens to be good at this

A TPU is built for one job

Google built its own chip because it could see how much of its datacentre was about to be doing one thing

A GPU

a general machine that
happens to be good at this

A TPU

a machine that does
this and little else

A TPU is built for one job

Google built its own chip because it could see how much of its datacentre was about to be doing one thing

A GPU

a general machine that happens to be good at this

A TPU

a machine that does this and little else

$256 \times 256 = 65,536$ multiply accumulate units in one grid

A TPU is built for one job

Google built its own chip because it could see how much of its datacentre was about to be doing one thing

A GPU

a general machine that happens to be good at this

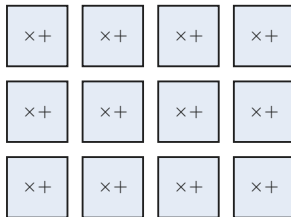
A TPU

a machine that does this and little else

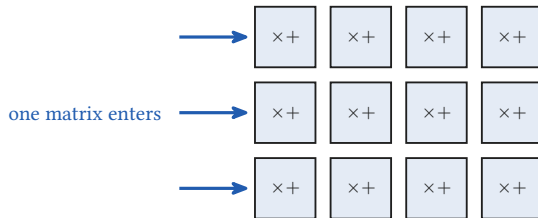
$256 \times 256 = 65,536$ multiply accumulate units in one grid

Jouppi et al., *In-Datacenter Performance Analysis of a Tensor Processing Unit*, ISCA 2017, [arXiv:1704.04760](#).

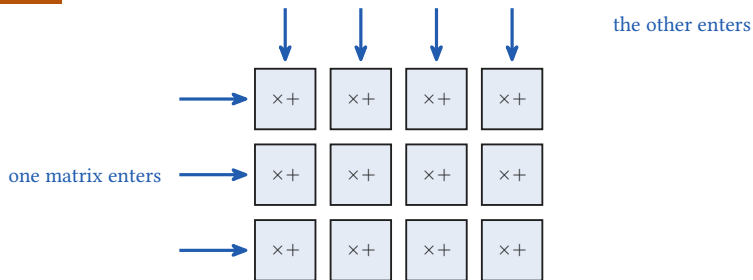
The systolic array: data walks through the grid



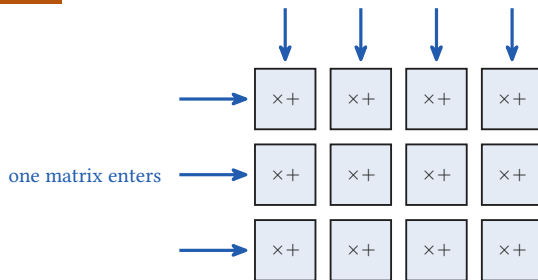
The systolic array: data walks through the grid



The systolic array: data walks through the grid



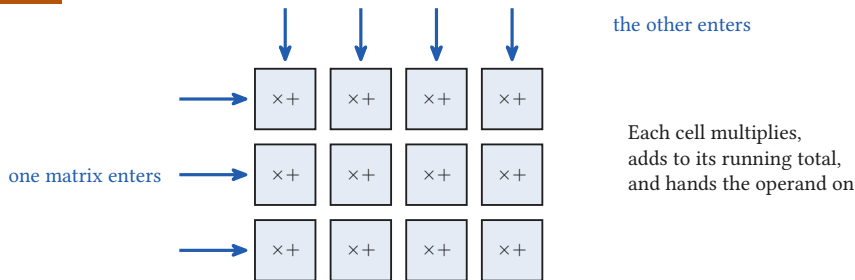
The systolic array: data walks through the grid



the other enters

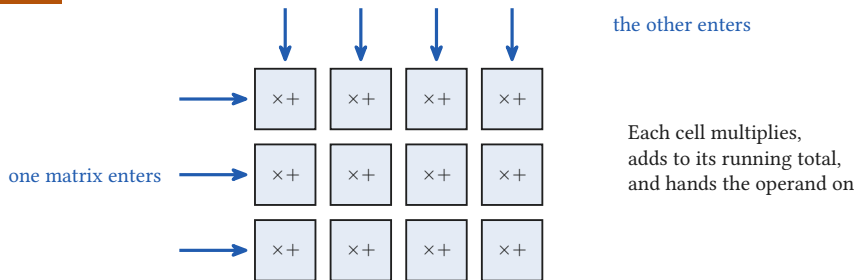
Each cell multiplies,
adds to its running total,
and hands the operand on

The systolic array: data walks through the grid



A value is read from memory once and then used by a whole row

The systolic array: data walks through the grid



A value is read from memory once and then used by a whole row

- That is the answer to the 246 operations per byte from 08:00: spend less on fetching, and the arithmetic stops waiting



Section 4

What it means for you

4

 What it means for you

Specialisation buys speed and costs freedom

The same trade, three times

Specialisation buys speed and costs freedom

The same trade, three times

CPU anything, slowly

Specialisation buys speed and costs freedom

The same trade, three times

CPU anything, slowly

GPU anything shaped like a matrix, very fast

Specialisation buys speed and costs freedom

The same trade, three times

CPU anything, slowly

GPU anything shaped like a matrix, very fast

TPU matrices, and almost nothing else, faster still

Specialisation buys speed and costs freedom

The same trade, three times

CPU anything, slowly

GPU anything shaped like a matrix, very fast

TPU matrices, and almost nothing else, faster still

Each step down the list gives up generality and gets throughput back

Specialisation buys speed and costs freedom

The same trade, three times

CPU anything, slowly

GPU anything shaped like a matrix, very fast

TPU matrices, and almost nothing else, faster still

Each step down the list gives up generality and gets throughput back

- Which is why your training loop runs on all three at once: Python on the CPU, the matrices on the tensor cores

What you can draw at 11:00

Many simple cores

What you can draw at 11:00



What you can draw at 11:00



What you can draw at 11:00



Three pictures, and the third one is the number you priced this morning

What you can draw at 11:00



Three pictures, and the third one is the number you priced this morning

- The box from 08:00 is open. A rate and a wattage, and now you know what produces them